

Rapport Databaser

Av Ann Johansson

Innehåll:

Sida 1: ER-Diagram

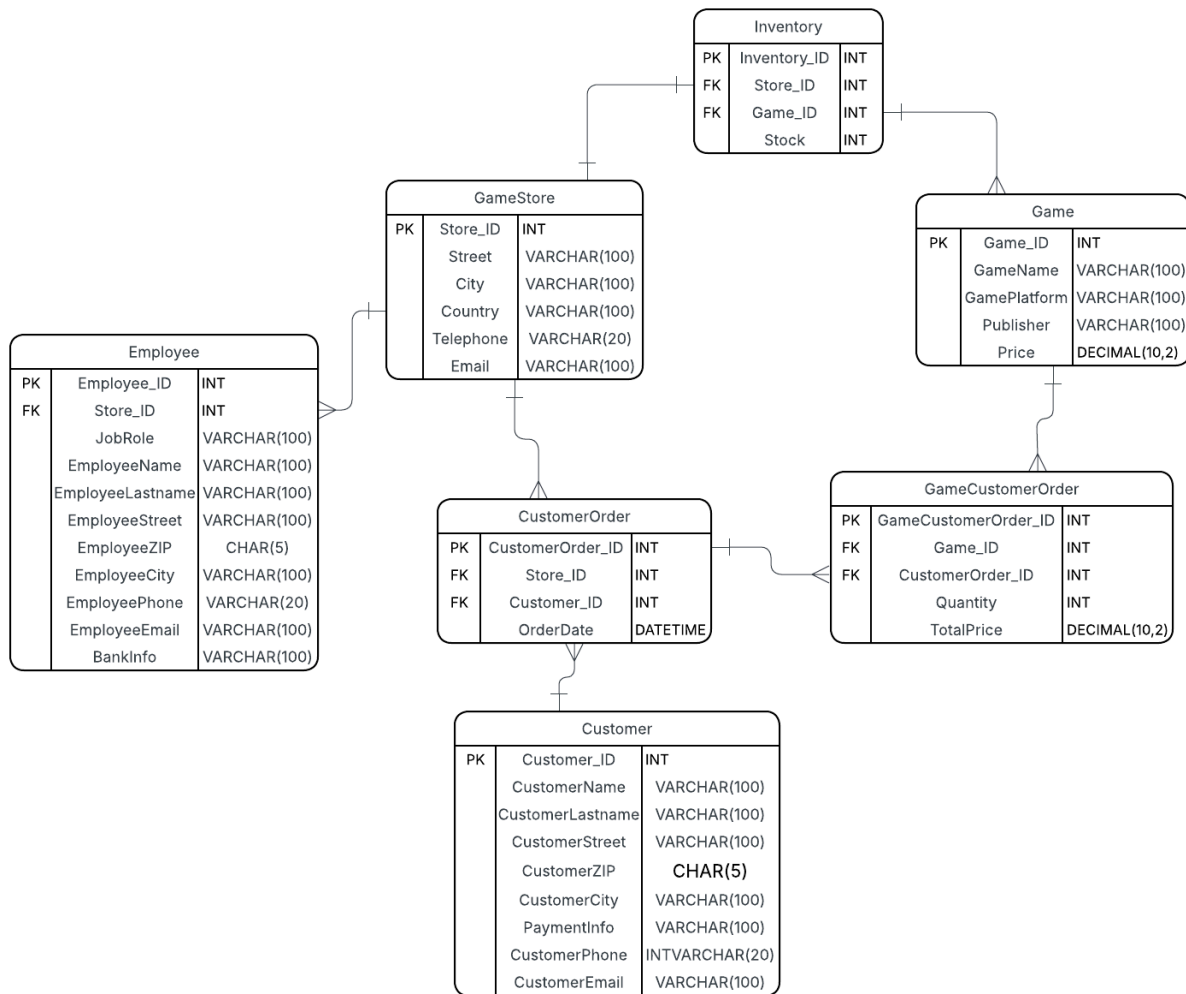
Sida 2: Förklaringar och motiveringar

Sida 3-8: Alla SQL-kommandon

Sida 9-10: Jämför SQL och LINQ

Sida 11: Säkerhet och Versionshantering

ER-Diagram



Förklaringar och motiveringar

Jag har valt att ha 7 tabeller som kan kopplas till en GameStore databas. Jag har tagit med butik, anställda, lager, produkt, order och kund, samt en tabell som hanterar produkt och order tabellernas many to many relation, så alla kunder skall kunna beställa flera spel i en order och alla spel skall kunna beställas av olika kunder samtidigt.

Kunder har en relation till order och det är ordern som knyter kunden samman med företaget som de beställer ifrån. Sen är det lagret som knyter samman produkterna med företaget. Jag har begränsat mig när det kommer till att skapa tabeller och hur stor databasen skall vara, annars skulle man kunna göra nya tabeller för [t.ex](#) de anställdas yrkesroller, kontaktinformation och spelinformation. Man kan fortsätta att dela in allt väldigt mycket, så det kan vara svårt att avgöra ibland vart man skall dra gränsen men det är väl där Normalisering kommer in.

I detta fall så har jag bestämt mig för att alla spelbutiker i denna databas är fysiska butiker. Annars skulle jag inte ha valt att ha NOT NULL på GameStore adress fälten. Jag har även valt att ha VARCHAR på telefonnummer och kontonummer så att det inte blir några fel som försvunna 0:or eller att man försöker lägga in tecken som bindestreck. Alla VARCHAR-fält har en begränsning på 100 tecken så att man inte skall kunna skriva in för mycket text, även om 100 tecken är rätt mycket, men det hindrar ändå en del från att någon skriver in onödigt mycket text. Telefonnummer har bara 20 tecken eftersom inga telefonnummer är längre, och postnummer är enbart 5 tecken. Jag valde att sätta IDENTITY(1,1) på PRIMARY KEY's så att de alltid får ett unikt ID nummer av sig själva och ID nummer är alltid INTEGER.

Alla SQL-kommandon

Create och Tables

```
CREATE DATABASE GameStoreDB;
```

```
USE GameStoreDB;
```

--As I see the game store as physical stores and not online stores I choose to have NOT NULL on the adress parts too.

```
CREATE TABLE GameStore (  
Store_ID INTEGER PRIMARY KEY IDENTITY(1,1),  
Street VARCHAR(100) NOT NULL,  
City VARCHAR(100) NOT NULL,  
Country VARCHAR(100) NOT NULL,  
Telephone VARCHAR(20) NOT NULL,  
Email VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE Game (  
Game_ID INTEGER PRIMARY KEY IDENTITY(1,1),  
GameName VARCHAR(100) NOT NULL,  
GamePlatform VARCHAR(100) NOT NULL,  
Publisher VARCHAR(100) NOT NULL,  
Price DECIMAL(10,2) NOT NULL  
);
```

```
CREATE TABLE Inventory (  
Inventory_ID INTEGER PRIMARY KEY IDENTITY(1,1),  
Store_ID INTEGER NOT NULL,  
Game_ID INTEGER NOT NULL,  
Stock INTEGER NOT NULL DEFAULT 0,  
FOREIGN KEY (Store_ID) REFERENCES GameStore(Store_ID),  
FOREIGN KEY (Game_ID) REFERENCES Game(Game_ID)  
);
```

-- Phone numbers will have VARCHAR instead of INTEGER so there won't be problems with '0' in the beginning or if other characters are put in like "-"

```
CREATE TABLE Customer (  
Customer_ID INTEGER PRIMARY KEY IDENTITY(1,1),  
CustomerName VARCHAR(100) NOT NULL,  
CustomerLastname VARCHAR(100) NOT NULL,  
CustomerStreet VARCHAR(100) NOT NULL,  
CustomerZIP CHAR(5) NOT NULL,
```

```
CustomerCity VARCHAR(100)NOT NULL,  
CustomerPhone VARCHAR(20) NOT NULL,  
PaymentInfo VARCHAR(100) NOT NULL,  
CustomerEmail VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE CustomerOrder (  
CustomerOrder_ID INTEGER PRIMARY KEY IDENTITY(1,1),  
Store_ID INTEGER NOT NULL,  
Customer_ID INTEGER NOT NULL,  
OrderDate DATETIME NOT NULL DEFAULT GETDATE(),  
FOREIGN KEY (Store_ID) REFERENCES GameStore(Store_ID),  
FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)  
);
```

```
CREATE TABLE GameCustomerOrder (  
GameCustomerOrder_ID INTEGER PRIMARY KEY IDENTITY(1,1),  
CustomerOrder_ID INTEGER NOT NULL,  
Game_ID INTEGER NOT NULL,  
Quantity INTEGER NOT NULL,  
TotalPrice DECIMAL(10,2) NOT NULL,  
FOREIGN KEY (CustomerOrder_ID) REFERENCES CustomerOrder(CustomerOrder_ID),  
FOREIGN KEY (Game_ID) REFERENCES Game(Game_ID)  
);
```

```
CREATE TABLE Employee (  
Employee_ID INTEGER PRIMARY KEY IDENTITY(1,1),  
Store_ID INTEGER NOT NULL,  
JobRole VARCHAR(100) NOT NULL,  
EmployeeName VARCHAR(100) NOT NULL,  
EmployeeLastname VARCHAR(100) NOT NULL,  
EmployeeStreet VARCHAR(100) NOT NULL,  
EmployeeZIP CHAR(5) NOT NULL,  
EmployeeCity VARCHAR(100) NOT NULL,  
EmployeePhone VARCHAR(20) NOT NULL,  
EmployeeEmail VARCHAR(100) NOT NULL,  
BankInfo VARCHAR(100) NOT NULL,  
FOREIGN KEY (Store_ID) REFERENCES GameStore(Store_ID)  
);
```

Inserts

```
INSERT INTO Customer (  

```

```

CustomerName, CustomerLastname, CustomerStreet, CustomerZIP, CustomerCity,
CustomerPhone, PaymentInfo, CustomerEmail)
VALUES
('Henrietta', 'Hansson', 'Hamngatan 1', '12345', 'Falköping', '070-1234567', 'VISA-9876',
'henny@penny.com'),
('Erik', 'Lindgren', 'Björnvägen 22', '45678', 'Göteborg', '072-9876543', 'Master-1122',
'erik.l@mail.se'),
('Olle', 'Persson', 'Kungsallén 5', '77788', 'Malmö', '076-1110099', 'Faktura',
'persson_o@online.net'),
('Maria', 'Jönsson', 'Åkersvägen 7', '65432', 'Uppsala', '070-0012345', 'Amex-4455',
'maria.j@uppsala.nu'),
('Viktor', 'Svensson', 'Solvägen 8', '90909', 'Luleå', '079-8877665', 'Klarna', 'viktor.s@norr.se');

```

```

INSERT INTO GameStore (
Street, City, Country, Telephone, Email)
VALUES
('Kungsgatan 12', 'Stockholm', 'Sweden', '08-1112233', 'sthlm@gamestore.se'),
('Lilla Torg 5', 'Malmö', 'Sweden', '040-998877', 'malmo@gamestore.se');

```

```

INSERT INTO Game (
GameName, GamePlatform, Publisher, Price)
VALUES
('Cyberpunk 2077', 'PC', 'CD Projekt RED', 499.50),
('Elden Ring', 'PS5', 'Bandai Namco', 599.00),
('The Sims 4', 'PC', 'Electronic Arts', 299.00),
('Call of Duty: Modern Warfare', 'XBOX', 'Activision', 699.00),
('Minecraft', 'Switch', 'Mojang', 199.90);

```

```

INSERT INTO Inventory (
Store_ID, Game_ID, Stock)
VALUES
(1, 1, 50),
(1, 2, 20),
(1, 3, 15),
(1, 4, 30),
(1, 5, 45),
(2, 1, 10),
(2, 2, 5),
(2, 3, 25),
(2, 4, 15),
(2, 5, 30);

```

```

INSERT INTO CustomerOrder (
Store_ID, Customer_ID)

```

VALUES

(1, 3),

(2, 5),

(1, 1),

(2, 4),

(1, 2);

INSERT INTO GameCustomerOrder (
CustomerOrder_ID, Game_ID, Quantity, TotalPrice)

VALUES

(1, 1, 1, 499.50),

(2, 2, 1, 599.00),

(2, 3, 2, 598.00),

(3, 5, 1, 199.90),

(4, 4, 1, 699.00),

(5, 1, 1, 499.50),

(5, 5, 1, 199.90);

INSERT INTO Employee (
Store_ID, JobRole, EmployeeName, EmployeeLastname, EmployeeStreet, EmployeeZIP,
EmployeeCity, EmployeePhone, EmployeeEmail, BankInfo)

VALUES

(1, 'Manager', 'Adam', 'Berg', 'Storgatan 1', '10111', 'Stockholm', '070-1112233',

'adam.b@game.se', 'Swedbank-999'),

(1, 'Sales Associate', 'Frida', 'Nilsson', 'Långa Gatan 3', '10111', 'Stockholm', '070-4445566',

'frida.n@game.se', 'Nordea-888'),

(2, 'Manager', 'Linda', 'Åkesson', 'Södergatan 10', '20222', 'Malmö', '076-7778899',

'linda.a@game.se', 'Handelsbanken-777');

Selects

-- Show all customers

SELECT * FROM Customer;

-- Show all employees first name and their job role

SELECT EmployeeName, JobRole

FROM Employee;

-- Shows all games that cost less than 500

SELECT * FROM Game

WHERE Price < 500;

-- Sorting all games in a descending order by their publishers

SELECT * FROM Game

```
ORDER BY Publisher DESC;
```

```
-- Choses all who have "ns" inside their last names
```

```
SELECT * FROM Customer
```

```
WHERE CustomerLastname LIKE '%ns%';
```

```
-- Counts all the games that are made for specific platforms (it is grouped by platforms)
```

```
SELECT COUNT(GameName) AS NumberOfGames, GamePlatform
```

```
FROM Game
```

```
GROUP BY GamePlatform
```

Joins

```
-- I JOIN the customer table and the orders to show which customers have made orders from  
which store and when
```

```
SELECT Customer.Customer_ID,
```

```
Customer.CustomerName,
```

```
Customer.CustomerLastname,
```

```
Customer.CustomerEmail,
```

```
CustomerOrder.Store_ID,
```

```
CustomerOrder.OrderDate
```

```
FROM Customer
```

```
JOIN CustomerOrder
```

```
ON Customer.Customer_ID = CustomerOrder.Customer_ID;
```

```
-- This joins 4 tables to show which games and how many of each game, customers have  
bought
```

```
SELECT C.Customer_ID, C.CustomerName, C.CustomerLastname, GCO.Quantity,
```

```
G.GameName
```

```
FROM Customer AS C
```

```
LEFT JOIN CustomerOrder AS CO
```

```
ON C.Customer_ID = CO.Customer_ID
```

```
LEFT JOIN GameCustomerOrder AS GCO
```

```
ON CO.CustomerOrder_ID = GCO.CustomerOrder_ID
```

```
LEFT JOIN Game as G
```

```
ON GCO.Game_ID = G.Game_ID;
```

Update

```
-- It's a sale and two of the games prices is going down by 150
```

```
UPDATE Game
```

```
SET Price = Price - 150
```

```
WHERE Game_ID IN (2, 4);
```


-- One customer have changed their phone number
UPDATE Customer
SET CustomerPhone = '012-345678'
WHERE Customer_ID = 3;

Delete

-- One of the customers have chosen to delete an order of a game
DELETE FROM GameCustomerOrder
WHERE GameCustomerOrder_ID = 3;

Jämför SQL och LINQ

SQL 1

```
SELECT EmployeeName, JobRole
FROM Employee;
```

LINQ jämförelse

```
var employeeInfo = Employee
    .Select(n => new { n.EmployeeName, n.JobRole })
    .ToList();
```

Förklaring 1

I databasen väljs två kolumner att visas från ett table som heter Employee genom att välja kolumnerna från ett table.

I LINQ för att vissa informationen behöver man använda query Select för att välja EmployeeName och JobRole och sen skriva in dem till en lista (ToList) för att kunna se vilka värden de innehåller. I mitt exempel är listan employeeInfo.

Likheterna är stora i Select delen mellan de båda, då det är där det väljs ut vad som skall väljas.

SQL 2

```
SELECT * FROM Game
ORDER BY Publisher DESC;
```

LINQ jämförelse

```
var orderDescending = Game
    .OrderByDescending(n => n.Publisher)
    .ToList();
```

Förklaring 2

Här i nästa query så ser man att man inte behöver använda Select för att specificera att man väljer alla i LINQ, men i båda använder vi oss av Order By och Descending där det specificeras att det är efter Publisher i Game som det ska sorteras. Återigen så behöver alltid värdet lagras i en ny lista i LINQ.

SQL 3

```
SELECT * FROM Customer
WHERE CustomerLastname LIKE '%ns%';
```

LINQ jämförelse

```
var reslut = Customer
    .Where(c => c.CustomerLastname.Contains("ns"))
    .ToList();
```

Förklaring 3

Här behövs inte Select specificeras i LINQ eftersom vi väljer alla men i Where raden påminner de om varandra, där man specificerar att det är i CustomerLastname som vi skall hitta värden som innehåller "ns". Skillnaden är att det heter LIKE i SQL och "Contains" i LINQ.

Säkerhet

Varför säker åtkomst till databaser är viktigt:

För att man ofta förvarar känslig information i databaser som inte obehöriga skall ha tillgång till då det kan orsaka stor skada.

Vad authentication och authorization betyder:

Det betyder att man validerar att användaren är den den utger sig för att vara och ger dem åtkomst och behörigheter till att ta del av och ändra data.

Hur man skyddar känslig data i ett backend projekt:

Man kan ha rollbaserad åtkomst till datan, man ska hålla databasen uppdaterad, kryptera lösenord genom att använda hashes, salt och peppers för att göra det svårare för hackare att komma åt dem, samt kryptera data med SSL/TLS vid transporter av den. Sen måste man alltid ha backup, använda brandvägg och vara noggrann med säkerheten Cloud leverantören erbjuder. Man skall även se till att använda åtgärder mot SQL injections när man skriver koden till databasen.

Versionshantering

Versionshantering är mycket viktig för databas utvecklingen då detta sparar alla versioner av koden ifall något händer med databasen. Särskilt då bara någon rad kod kan ta bort eller ändra stora bitar data som skulle vara försvunnen annars. Det är också bra för samarbete, så att flera personer kan ta hand om och arbeta på den med commits som berättar vad som gjorts.